

LAB REPORT

Linux Memory Segments and GDB Inspection

Name: Ayusha Sapkota

Module: Operating Systems

Lab: Memory Segments and GDB Inspection

Operating System: Ubuntu Linux

Architecture: x86-64

Compiler: GCC

Debugger: GDB

Table Of contents

1. Introduction.....	3
2. Source Code.....	4
Figure 1: C Program Implementation for Memory Segment Layout Verification.....	4
3. Compilation.....	4
Figure 2: Compilation and GDB Debugger Initialization for demo2.c.....	4
4. Running the Program in GDB.....	5
Figure 3: Initializing GDB Session and Loading Debug Symbols.....	5
Figure 4 :Program Execution in GDB Displaying Memory Segment Addresses.....	5
5. Runtime Address Analysis.....	6
6. Inspecting ELF Sections.....	7
Figure 6: Executing info file Command in GDB to Display Memory Section Headers.....	7
Figure 7: Memory Address Ranges for .text, .data, and .bss Sections.....	8
7. Inspecting the Global Variable.....	9
Figure 8: Inspecting Global Variable Address and Memory Value in GBD.....	9
Figure 9: Output Analysis Confirming Global Variable Data Section Assignment.....	9
8. Inspecting the String Literal.....	10
Figure 10 : Insepting String Literal Pointer Address using GDB print Command.....	10
Figure 11: Examining String Content at Memory Address using GDB x/s Command.....	10
Figure 12: Checking Pointer Variable Memory Address using GDB print Command.....	10
9. Endianness.....	11
10. Why Are .text and .rodata Separate?.....	11
11. Instruction Pointer and .text.....	12
12. Conclusion.....	12
Questions and Answers.....	13

1. Introduction

The purpose of this laboratory is to understand how a compiled C program is organized in memory. The experiment focuses on the main memory sections of an executable program:

text – contains executable machine instructions.

rodata – contains read-only data such as string literals.

data – contains initialized global and static variables.

bss – contains uninitialized global and static variables.

The GNU Debugger (GDB) was used to examine the runtime addresses of these sections and inspect the contents stored in memory.

2. Source Code

The following C program was used for the experiment:

```
#include <stdio.h>

// 1. Data Section (Initialized global data)
int data_global_var = 0x12345678;

// 2. RoData Section (Read-only string constant literal)
const char *text_string_literal = " Text_Segment_Verification";

// 3. Text Section (Function execution code block)
void check_execution() {
    printf("Executing within the text segment instruction block.\n");
}

int main() {
    printf("Addresses:\n");
    printf("Data (Global Initialized): %p\n", (void*)&data_global_var);
    printf("Text String (ROData): %p\n", (void*)text_string_literal);
    printf("Text Code (Function): %p\n", (void*)&check_execution);

    check_execution();

    // Breakpoint here to inspect text and data layers
    printf("Ready for Text/Data segment inspection!\n");
    return 0;
}
```

Figure 1: C Program Implementation for Memory Segment Layout Verification

3. Compilation

The program was compiled using GCC with debugging information enabled. The `-g` option includes debugging information so that GDB can associate machine instructions with the original C source code.

The generated executable was verified using `ls`.

```
root@AYUSHA:~/lab5# ls
demo2.c
root@AYUSHA:~/lab5# gcc -g demo2.c -o demo2
root@AYUSHA:~/lab5# ls
demo2 demo2.c
root@AYUSHA:~/lab5# gdb ./demo2
```

Figure 2: Compilation and GDB Debugger Initialization for demo2.c

4. Running the Program in GDB

The executable was opened using GDB.

```
root@AYUSHA:~/lab5# gdb ./demo2
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo2...
```

Figure 3: Initializing GDB Session and Loading Debug Symbols

A breakpoint was placed at line 23, Where GDB reported:

Breakpoint 1 at 0x11f9: file demo2.c, line 23.

The program was then executed by run function. The program produced the following addresses:

Data (Global Initialized): 0x555555558010

Text String (ROData): 0x555555556008

Text Code (Function): 0x55555555169

Executing within the text segment instruction block.

The program stopped at the breakpoint: Breakpoint 1, main () at demo2.c:23

```
(gdb) break 23
Breakpoint 1 at 0x11f9: file demo2.c, line 23.
(gdb) run
Starting program: /root/lab5/demo2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized): 0x555555558010
Text String (ROData): 0x555555556008
Text Code (Function): 0x55555555169
Executing within the text segment instruction block.

Breakpoint 1, main () at demo2.c:23
23      printf("Ready for Text/Data segment inspection!\n");
```

Figure 4 :Program Execution in GDB Displaying Memory Segment Addresses

5. Runtime Address Analysis

When executing a program, the operating system assigns different components to specific memory regions:

I..text **section** (0x55555555169): Stores executable function instructions like `check_execution()`.

II..rodata **section** (0x555555556008): Holds fixed, read-only string constants like `"Text_Segment_Verification"`.

III..data **section** (0x555555558010): Contains initialized global variables like `data_global_var` that can be read and modified during runtime.

6. Inspecting ELF Sections

The following GDB command was used:

```
(gdb) info file
Symbols from "/root/lab5/demo2".
Native process:
  Using the running image of child process 1951.
  While running this, GDB does not access memory from...
Local exec file:
  '/root/lab5/demo2', file type elf64-x86-64.
  Entry point: 0x555555555080
  0x0000555555554350 - 0x0000555555554374 is .note.gnu.build-id
  0x0000555555554374 - 0x0000555555554390 is .interp
  0x0000555555554390 - 0x00005555555543b4 is .gnu.hash
  0x00005555555543b8 - 0x0000555555554478 is .dynsym
  0x0000555555554478 - 0x000055555555450c is .dynstr
  0x000055555555450c - 0x000055555555451c is .gnu.version
  0x0000555555554520 - 0x0000555555554550 is .gnu.version_r
  0x0000555555554550 - 0x0000555555554628 is .rela.dyn
  0x0000555555554628 - 0x0000555555554658 is .rela.plt
  0x0000555555555000 - 0x000055555555501b is .init
  0x0000555555555020 - 0x0000555555555050 is .plt
  0x0000555555555050 - 0x0000555555555060 is .plt.got
  0x0000555555555060 - 0x0000555555555080 is .plt.sec
  0x0000555555555080 - 0x000055555555520f is .text
  0x0000555555555210 - 0x000055555555521d is .fini
  0x0000555555555600 - 0x000055555555560e8 is .rodata
  0x000055555555560e8 - 0x00005555555556124 is .eh_frame_hdr
  0x00005555555556128 - 0x000055555555561f4 is .eh_frame
  0x000055555555561f8 - 0x00005555555556228 is .note.gnu.property
  0x00005555555556228 - 0x00005555555556248 is .note.ABI-tag
  0x00005555555557db0 - 0x00005555555557db8 is .init_array
  0x00005555555557db8 - 0x00005555555557dc0 is .fini_array
  0x00005555555557dc0 - 0x00005555555557fb0 is .dynamic
  0x00005555555557fb0 - 0x00005555555558000 is .got
  0x00005555555558000 - 0x00005555555558020 is .data
  0x00005555555558020 - 0x00005555555558028 is .bss
  0x00007ffff7fc02a8 - 0x00007ffff7fc02cc is .note.gnu.build-id in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc02d0 - 0x00007ffff7fc0408 is .hash in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc0408 - 0x00007ffff7fc0564 is .gnu.hash in /lib64/ld-linux-x86-64.so.2
  0x00007ffff7fc0568 - 0x00007ffff7fc0910 is .dynsym in /lib64/ld-linu--Type <RET> for more, q to quit, c to continue without paging--q
```

Figure 6: Executing `info file` Command in GDB to Display Memory Section Headers

GDB identified the executable as: file type elf64-x86-64

This means the executable uses the ELF (Executable and Linkable Format) and is a 64-bit x86-64 executable.

The relevant memory sections were:

START AND END ADDRESSES OF MEMORY SECTIONS			
	Section	Start Address	End Address
1	.TEXT	0X000055555555080	0X00005555555520F
2	.DATA	0X0000555555558000	0X0000555555558020
3	.BSS	0X0000555555558020	0X0000555555558028

Figure 7: Memory Address Ranges for .text, .data, and .bss Sections

7. Inspecting the Global Variable

The address of `data_global_var` was obtained using:

```
(gdb) print &data_global_var  
$1 = (int *) 0x555555558010 <data_global_var>
```

Figure 8: Inspecting Global Variable Address and Memory Value in GDB

The value stored at this address was inspected using:

```
(gdb) x/wx &data_global_var  
0x555555558010 <data_global_var>: 0x12345678
```

Figure 9: Output Analysis Confirming Global Variable Data Section Assignment

This confirms that the global variable contains: 0x12345678

And is located at: 0x555555558010

Since this address falls within the `.data` range, the variable is correctly located in the `.data` section.

8. Inspecting the String Literal

The string pointer was inspected using:

```
(gdb) print &text_string_literal  
$3 = (const char **) 0x55555558018 <text_string_literal>
```

Figure 10 : Inspecting String Literal Pointer Address using GDB print Command

The string itself was then examined using:

```
(gdb) x/s text_string_literal  
0x55555556008: " Text_Segment_Verification"
```

Figure 11: Examining String Content at Memory Address using GDB x/s Command

This confirms that the actual string is stored at:0x55555556008 which falls within the .rodata section.

The address of the pointer variable itself was also checked:

```
(gdb) print &text_string_literal  
$3 = (const char **) 0x55555558018 <text_string_literal>
```

Figure 12: Checking Pointer Variable Memory Address using GDB print Command

This demonstrates an important distinction:

- I. 0x55555556008 → address of the actual string in .rodata
- II. 0x55555558018 → address of the pointer variable in .data

9. Endianness

The variable was initialized with: 0x12345678

On an x86-64 system, memory uses **Little Endian** byte ordering. Therefore, the bytes are stored from the least significant byte to the most significant byte:

Value:0x12345678

Memory:0x78 0x56 0x34 0x12

Thus, the least significant byte 0x78 is stored at the lowest memory address.

10. Why Are .text and .rodata Separate?

Although both .text and .rodata are generally read-only, they have different purposes.

.text

The .text section contains executable machine instructions. The processor fetches instructions from this region for execution. Therefore, it is normally given permissions similar to: Read + Execute

.rodata

The .rodata section contains constant data such as string literals. It should be readable but should not be executable. Its permissions are normally similar to:Read only

Keeping these sections separate improves security because constant data does not need execute permission. This helps prevent data such as strings from being executed as machine instructions.It also allows the operating system's memory-management system to apply appropriate permissions to different memory pages.

11. Instruction Pointer and .text

The program was compiled for the **x86-64 architecture**. On x86-64, the instruction pointer register is called: RIP. It contains the address of the next instruction to be executed.

The `check_execution()` function is located at: `0x55555555169`

This address is inside the `.text` section: `0x000055555555080 - 0x00005555555520f`

Therefore, the function is correctly located inside the executable code section.

12. Conclusion

This laboratory demonstrated how a compiled C program is divided into different memory sections. Using GCC and GDB, the addresses and contents of the `.text`, `.rodata`, `.data`, and `.bss` sections were inspected.

The experiment confirmed that:

- 1.Executable functions are stored in `.text`.
- 2.Constant strings are stored in `.rodata`.
- 3.Initialized global variables are stored in `.data`.
- 4.Uninitialized global/static data is stored in `.bss`.
- 5.The system uses **x86-64 architecture**.
- 6.The system uses **Little Endian** byte ordering.
- 7.GDB can be used to inspect runtime addresses, memory contents, ELF sections, and machine instructions.

Overall, the experiment provided practical understanding of how the operating system and compiler organize a program in memory and how GDB can be used to inspect these memory regions.

Questions and Answers

Q1. Which memory section contains the initialized global variable?

The initialized global variable `data_global_var` is stored in the `.data` section.

Q2. Where is the string literal stored?

The string literal "Text_Segment_Verification" is stored in the `.rodata` section because it is read-only constant data.

Q3. Which section contains executable instructions?

Executable machine instructions are stored in the `.text` section.

Q4. What architecture is the executable?

The executable is: ELF 64-bit x86-64

Therefore, it is a 64-bit x86-64 Linux executable.

Q5. What is the endianness of the system?

*The system uses **Little Endian** byte ordering.*

The value: 0x12345678 is stored as: 78 56 34 12 in memory.

Q6. Why are `.text` and `.rodata` separated?

They are separated because they have different purposes and memory permissions. `.text` contains executable instructions and normally requires read and execute permissions, while `.rodata` contains constant data and only needs read permission. Separating them improves memory protection and security.